

NAME:VIJAY SURIYA P
SRN:PES1UG24AM320
SEC:F

<https://chatgpt.com/c/6ac5bb28-52e8-83ec-a31c-065b69834a70>

The **main error is in `logic.py`, line 5:**

```
partial = sum(ch in code for ch in guess) - exact
```

```
logic
```

Why this is wrong

`ch in code` only checks whether the symbol exists **somewhere** in the code. It does **not** track how many times that symbol has already been used.

For Mastermind, **each occurrence in the secret code can be counted only once**, and exact matches must be counted first. Your README explicitly requires this duplicate-aware behavior. README

Example

Suppose:

Code: 1123
Guess: 1111

Your code calculates:

```
exact = 2
```

Then:

```
sum(ch in code for ch in guess)
```

checks each `1` independently:

```
1 in 1123 → True  
1 in 1123 → True  
1 in 1123 → True  
1 in 1123 → True
```

So:

```
sum = 4  
partial = 4 - 2 = 2
```

But the correct answer is:

Exact = 2
Partial = 0

The two 1s in the code have already been consumed by the two exact matches. The remaining guessed 1s cannot match anything.

What you need to change

You need an algorithm that:

1. Finds all **exact matches** first.
2. Removes/marks those matched positions.
3. Checks the remaining unmatched guesses against the remaining unmatched code symbols.
4. Ensures each code occurrence is counted **at most once**.

Your `game.py` itself is calling the feedback function correctly:

```
exact, partial = feedback(self.code, guess)
```

game

So the **actual bug is in the duplicate-handling logic**, not the game loop.

Also, according to your README, Tasks 2–4 are currently not implemented yet: difficulty modes, readable history, and some complete win/loss/input lifecycle handling are still missing. README

I can show you the **exact corrected `logic.py` code** while keeping your existing project structure.

how should i go about fixing it

The safest way is to **fix Task 1 first**, test it, and only then move to Tasks 2–4. Your README specifically says exact matches must be resolved before partial matches, with each code position counted at most once. README

Step 1: Understand the two-pass logic

Think of the feedback as two stages.

Pass 1 — exact matches

For:

Code: 1123
Guess: 1131

Compare position by position:

1 = 1 → exact
1 = 1 → exact
2 ≠ 3
3 ≠ 1

So:

exact = 2

Now those first two positions are already **used**.

Pass 2 — partial matches

Only compare the unmatched positions:

Code remaining: 2, 3

Guess remaining: 3, 1

3 exists in the remaining code, so:

partial = 1

Final:

Exact = 2

Partial = 1

Step 2: Replace the incorrect line

Your current code does:

```
exact = sum(a == b for a, b in zip(code, guess))
partial = sum(ch in code for ch in guess) - exact
```

The second line is the bug because it doesn't keep track of **which occurrences have already been consumed**. logic

A simple way to fix it is to use two lists of unmatched symbols:

```
def feedback(code, guess):
    exact = 0
    remaining_code = []
    remaining_guess = []
    # First: find exact matches
    for c, g in zip(code, guess):
        if c == g:
            exact += 1
    else:
        remaining_code.append(c)
        remaining_guess.append(g)
    # Second: find partial matches
    partial = 0
    for g in remaining_guess:
        if g in remaining_code:
            partial += 1
            remaining_code.remove(g)
    return exact, partial
```

Step 3: Why **remove()** matters

This is the important part:

```
remaining_code.remove(g)
```

Suppose:

Code remaining: 1 2 3

Guess remaining: 3 3 1

First 3 matches:

partial = 1

remaining_code = [1, 2]

The second 3 now **cannot** match again because there is no 3 left.

That's exactly what your current implementation fails to enforce.

Step 4: Test it before changing anything else

Run these cases:

Code	Guess	Expected
1123	1111	2 exact, 0 partial
1123	1131	2 exact, 1 partial
1234	4321	0 exact, 4 partial
1234	5555	0 exact, 0 partial
1111	1111	4 exact, 0 partial

Your README specifically asks you to test repeated symbols, no matches, and mixed exact/partial cases. README

Step 5: Don't change `game.py` yet

Your `game.py` is already calling:

```
exact, partial = feedback(self.code, guess)
```

so the feedback function is the right place to fix Task 1. game

Once this works, move on in this order:

Task 1 → Task 2 → Task 3 → Task 4

That makes it much easier to identify which change causes a bug.